Further Lean proof simplifications after prior proposals

DiophantineConsistency third review

26 October 2026 Pinned source `22e020937caa81d0cb72a4f63071d6c0bcc1eba9`

Recommendation. Test five small proof replacements in the new machine and row modules first. Consider one larger change separately: describe the two horizontal edges of a constructed row by one spatial pulse. The latter makes compatibility a substitution, but does *not* reduce nonblank lines once its helper is included.

This review treats the previous proposals as completed, including their advertised adapters and design-only APIs. The six ideas below address obligations left *after* that virtual refactoring. They do not repackage compiler residuals, generated degree certificates, scanner iteration, typed decoders, or boundary views.

ID	Remaining proof obligation	Incremental change
E1	Injectivity of the actual state number with halt at 215	18 to 7 lines; one extra import
M1	Reflect action halt through the injective state number	10 to 3 declaration lines
M2	Compute color after a one-cell tape write	15 to 11 declaration lines
M3	Convert an absolute-value trace bound to exterior cases	8 to 3 declaration lines
R1	Propagate a predicate through an open integer interval	31 to 23 declaration lines
R2	Match horizontal edges of the constructed step row	One shared edge invariant; 28 to 29 lines

What changed upstream. Between the preceding review pin and this pin, the 32 modified active Lean files shrink by 1,502 physical lines in aggregate. Another 34 files add 3,479 lines, so the net active-source change is **plus 1,977 lines**. Most additions bind the literal machine and tile language to actual step and row semantics. Those are valuable new results, not a failed simplification, and none of the observed savings is credited to this report.

What is checked. The source inventory and dependency evidence are hash verified; proposed patches and metrics receive static checks. **None of the new Lean has been compiled.** No Lean or Lake runtime is installed in the review environment. Compilation, kernel axiom comparisons, downstream checks, and performance remain open gates. The PDF and source artifacts are research proposals, not an adoption claim.

Suggested order. Start with M3, M1, and M2, which introduce no helpers or imports. Try R1 with its unchanged theorem interface. Evaluate E1 against the cost of one Mathlib import. Treat R2 as an optional API normalization rather than a line-saving patch.

The next pages separate observed commits, assumed prior outcomes, exact candidates, and validation. Source links are pinned; artifact paths are relative to the accompanying archive. No repository source was edited or published.

1 Recent commits and the source boundary

The comparison starts at `b310142330df8` and ends at `22e020937caa8`, timestamped 26 October 2026 at 16:54:45 UTC. Its 27 commits form a Git DAG difference, not a claim that development proceeded in a single chronological chain.

1.1 Already implemented proof reductions

The current tree contains `CompilerSupport.lean`; the two DAG quartic backends and quadratic register system delegate finite-residual facts to it. It also contains `TotalDegreeBound.lean`; the ShortSystem, ShortFrontier, and ShortReducedStrong files use the total-degree tactic instead of large generated degree proofs. These are baseline facts, not new recommendations.

The same interval includes reuse of population/coefficient bounds, polynomial specialization, positive-domain transport, signed Pell facts, primitive-recursion equations, natural-assignment maps, optional list lookup, unconditional weak-cap bounds, and mask-support arithmetic. The upstream adoption review reports selected compilation checks and distinguishes queued rebuilds. Those are *upstream reports*, not tests reproduced here. See the upstream review and adoption record.

1.2 New semantics that leave new local proof work

Commit	Relevant development
`90a2ded65d09`	Actual first machine step and configuration-color semantics
`edf3a19d0cc5`	Exact tile inputs and uniform finish rows from one event
`993d6730a283`	Operational color updates and receiver targets
`04267333bd04`	Strict strip head containment and sharp actual trace bounds
`a421ec86eb86`	Actual row evolution and construction of the reverse step row

There are sixteen new semantic modules with sixteen paired Checks modules, plus the two shared proof-support modules. The new candidates preserve the actual decimal machine, the 942-tile language, live versus halted state interpretation, write-before-move semantics, and strict interval boundaries.

1.3 Coverage that does not depend on a capped comparison

GitHub's repository-wide comparison returned exactly 300 files. It is capped and is not a complete repository-change inventory. Instead, the untruncated old and new Lean trees contain 574 and 608 files. Comparing paths and Git blob identities gives **66 changed active Lean paths: 34 added and 32 modified, with none removed**. This independently matches all active Lean entries in the comparison response.

The 860-file source inventory was materialized at the pin: 782 files were reused only after matching their current Git blob identities, and 78 were fetched afresh. Five directly imported BusyBeaver files are separately recorded. All supplied-byte checks pass. This is a selected source snapshot, *not* a complete checkout or build dependency closure. The complete timeline, tree difference, counts, and verification scripts are in `provenance/`.

2 The assumed applied baseline

The baseline is a union of promised outcomes, not a pile of patches. Several earlier packets offer competing implementations of the same result. Applying them literally together would conflict. For this review, their strongest compatible outcome is available: a shared theorem, a usable adapter, or a completed refactoring with the original semantic guards. A design-only proposal is not fresh merely because its implementation was never supplied.

The audit binds six placed packets in the repository research directory, five fresh incoming archives, and the nine recommendations from the preceding delivered study. Its 75 legacy rows, 31 additional or strengthened incoming contracts, and nine previous-study rows *overlap*; 115 ledger rows do not mean 115 independent improvements. The audit checks 321 current placed blobs and rehashes all eleven archive streams.

2.1 Prior outcomes removed before looking for new ideas

Ledger	Already usable outcome	Work no longer eligible for new credit
V01–02	Exact decoders, lookup, scanner and history APIs	Repeated parsing, Option failure splits, Nat iteration and typed/encoded simulation
V03–05	Witness layouts, algebraic finalizers, inverse graphs	Assignment extensionality, coordinate packaging, divMod uniqueness, pairing inverses
V06–09	Residuals, degree/cost interfaces, carriers, coefficient and DAG certificates	Generated gate proofs, common tails, sparse-row checks, generator factoring
V10–11	Cyclic arithmetic, carry facts and canonical words	Modulus transport, support sums, carry recurrence and padding witnesses
V12	Finite tile data and controller views	Direction descriptors, finite dispatch, local controller invariants, high north views
V13–15	Computability, object semantics and module boundaries	Graph-builder duplication, bridge packaging, unrestricted induction substitutes, thin import facades
V16–18	Pell signs, resource budgets, spatial and word boundaries	Sign chasing, depth-adapter implementation, sparse placement, word congruence and known prefix offsets

The full audit is in `baseline_audit/`. Its `ASSUMED_APPLIED_BASELINE.md` and `ASSUMPTION_LEDGER.json` give the API contracts, exact prior-section citations, and per-proposal obligations.

2.2 Three distinctions keep the review honest

The adoption record and the pin are distinct: the record mentions queued `NatIterate` work, but no such module is present at the pin. Its outcome still belongs to the virtual baseline. A broad library-reuse suggestion does not reserve every later client, whereas an advertised adapter does. Finally, local controller and north-color views do not imply compatibility of neighboring positions in a newly constructed row.

3 Novelty and incremental benefit

All six candidates operate in modules newly added since the previous pin, but age alone is not the novelty test. Each must also survive the virtual APIs on the preceding page.

ID	Closest assumed prior outcome	Obligation that still remains
E1	V05 inverse-coordinate graphs; V01 Option injectivity	Identify the actual none-at-215 state map with the *last*-slot finite equivalence, then compose injectivity
M1	V12 finite controller invariants; V02 simulations	Reflect equality for this action adapter through its already injective state code, retaining the target bound
M2	V02 tag dispatch and V01 constructor views	Exploit the one-cell *write support* so destination classification need not be split in this color proof
M3	V12 controller certificates; existing library reuse	Convert the new sharp trace bound through the exact ordered-group `le_abs'` equivalence
R1	V02 Nat iteration; V12 cycle propagation	Translate an arbitrary open integer interval to an integer induction motive, removing both toNat endpoint conversions
R2	V12 rail descriptors and NorthView; V18 word boundaries	Derive both spatial edges from the actual tile constructors as evaluations of one bond-indexed function

3.1 What the measurements mean

The three machine replacements retain their public theorem headers byte for byte. Together they change 33 to 17 nonblank declaration lines, or 22 to 6 proof-body lines: **16 lines removed with no helper or import overhead**. R1 changes 31 to 23 declaration lines, or 26 to 18 body lines, also without an external helper. These are measurements against pinned source text, not a built virtual checkout. The ledger argues that the same obligations survive prior outcomes; it does not generate or compile a combined earlier-patch tree.

E1 changes 18 to 7 declaration lines, or 17 to 6 proof-body lines. Its local identity proof is included. One explicit Mathlib import is charged separately: 11 fewer theorem lines become 10 fewer module lines.

R2 is deliberately reported differently. Its replacement span has **28 to 29 nonblank code lines**, including the pulse definition, paired proof, projections, and horizontal client. It decreases the span's bytes from 1,912 to 1,553, but that is not evidence of less kernel work. The benefit is one shared case proof and an explicit spatial invariant; old west/east statement shapes are normalized, so this is not a pure proof-body edit.

3.2 Claims not made

There is no measured compile-time improvement, proof-term compression, or new axiom audit. Physical source lines are a readability proxy, not proof complexity. No combined percentage is offered: it would mix proof-only edits, an added import, and an API-changing alternative. Larger abstractions without a demonstrated remaining consumer are deferred rather than added to the recommendation count.

4 Reflect state information through existing injectivity

4.1 E1 Use the equivalence that puts halt last

Source anchors. `WangMachineEncoding.lean` lines 20–51. The current `stateNumber_injective` compares both Options, uses bounds to exclude the two mixed cases, and applies `Fin.ext` in the live/live case. The executable map is exactly the inverse of Mathlib's `finSuccEquivLast`, followed by `Fin.val`.

```
theorem stateNumber_injective : Function.Injective stateNumber := by
  have hcode : stateNumber = fun s : Option (Fin 215) =>
      ((finSuccEquivLast.symm s : Fin 216) : ℕ) := by
    funext s
    cases s <;> simp [stateNumber]
  rw [hcode]
  exact Fin.val_injective.comp finSuccEquivLast.symm.injective
```

Add `import Mathlib.Logic.Equiv.Fin.Basic`. The pinned library defines this equivalence and its simp laws for `some` and `none` in Mathlib lines 186–204. The source and its Git blob/hash binding are supplied in `encoding/evidence/`.

Why this is safe in intent. `stateNumber` itself stays unchanged: `none` is 215 and `some q` is `q.val`. The ordinary zero-slot `finSuccEquiv` is *not* a substitute; it would encode the sentinel differently. This is proof reuse for the actual concrete adapter, not a redesign of state encodings or another generic inverse-graph API.

Tradeoff and gate. The local equality and composed injectivity must elaborate under the pinned toolchain. Check direct import cost and compare `#print axioms`; do not assume the use of `funext` and `simp` preserves the old axiom footprint. The candidate includes the actual sentinel and live-state regression statements.

4.2 M1 Reflect action halt through that map

Source anchors. `WangMachineAction.lean` lines 60–69. This proof needs only the *existing* injectivity theorem; it does not depend on accepting E1. Keep its original statement and replace its two-direction equality transport by:

```
  rw [← stateNumber_injective.eq_iff,
    act_stateNumber a ha, stateNumber_none]
```

Reflecting equality through `stateNumber` gives the encoded equality. The action lemma identifies the next-state code with the target, and the sentinel lemma identifies halt with 215. The complete patch uses one physical proof line; it is wrapped above only for readability.

Preserve the premise. The bound `ha : a.2.2 < 216` is essential. The raw adapter also maps an invalid target of 216 to `none`; dropping the bound would make the equivalence false. No action table, decoder theorem, direction-specific target set, or terminal convention changes. This is a 10-to-3-line declaration replacement, with no new definition, import, or helper.

5 Reduce local machine case analysis

5.1 M2 Split on the write site only

Source anchors. `WangMachineStep.lean` lines 62–76. The theorem's result distinguishes the moved head, old head, and other cells. Its proof need not branch on all three. A write affects one cell. Away from that cell, the tape frame rule holds whether or not the position is the moved head; leave that destination test symbolic on both sides.

```
by_cases ho : x = cfg.head
· subst x
  simp [color, (moved_head_ne a.move cfg.head).symm]
· simp [color, ho, Tape.read_write_of_ne cfg.tape ho a.write]
```

At the old head, movement ensures that the old position is not the new head, and `Tape.read_write_same` resolves the write. Everywhere else, `Tape.read_write_of_ne` removes the write without deciding the remaining destination conditional. Two decisions and three leaves become one decision and two leaves; the three-way conclusion is unchanged. The declaration changes from 15 to 11 nonblank lines.

Semantic checks. An action entering `none` still writes and moves. At the new head, the read is the old tape bit; its halt color is 432 or 433. The proof uses tape reads rather than normalized list equality and therefore does not silently assume a duplicate-free tape representation. The patch adds no global simp rule.

5.2 M3 Use the exact absolute value view

Source anchors. `WangMachineTraceBounds.lean` lines 89–96. The current proof splits on the sign of `x`, rewrites `|x|`, and reconstructs the two exterior cases. Mathlib already supplies exactly the required disjunction:

```
le_abs' : a ≤ |b| ↔ b ≤ -a ∨ a ≤ b
```

The complete replacement body is:

```
exact run_read_outside M t x (le_abs'.mp hx)
```

The library theorem is produced by `@[to_additive]` from `le_mabs'` in the pinned absolute-value source, lines 66–67. The source evidence is supplied with the candidate; this verifies the declaration's provenance, not its elaboration at this call site. No import is added because the module already imports the relevant library.

The threshold remains inclusive: $t \le |x|$, including $t = 0$ and $x = 0$. There is no new positivity premise and no change to write-before-move support or stationary halted runs. The declaration changes from eight to three nonblank lines. The underlying trace induction is left alone; a new envelope structure would add overhead without an established additional saving.

Artifacts. `machine/MachineCandidates.lean` contains isolated source-exact statements and edge-case checks. The three independent patches and complete proposed modules are accompanied by a deterministic metric builder and static validation record.

6 R1 Induct over a translated integer interval

Source anchors. `WangTiles942ResetRows.lean` lines 41–73. The existing theorem proves rightward and leftward propagation by separate natural inductions, then chooses a direction and converts a distance with `Int.toNat`. Use one motive over an integer displacement instead. Retain the original statement, including strict boundaries and both local propagation premises.

```
have hshift (z : ℤ) : l < x₀+z → x₀+z < r → P (x₀+z) := by
  induction z using Int.induction_on with
  | zero =>
      intro hl hr
      simpa only [add_zero] using h₀
  | succ n ih =>
      intro hl hr
      have hs := hnext (x₀+(n : ℤ)) (by omega) (by omega)
        (ih (by omega) (by omega))
      convert hs using 1 <;> omega
  | pred n ih =>
      intro hl hr
      have hs := hprev (x₀+(-(n : ℤ))) (by omega) (by omega)
        (ih (by omega) (by omega))
      convert hs using 1 <;> omega
intro x hl hr
have he : x₀+(x-x₀) = x := by omega
simpa only [he] using hshift (x-x₀) (by omega) (by omega)
```

Why the motive includes both bounds. In a positive step, the induction hypothesis applies to the point one cell before the target; in a negative step, it applies to the point one cell after it. The fixed interior seed supplies the missing bound in each direction. The two bounds must remain implications in the induction motive, not be fixed prematurely as hypotheses about the final target.

At the end, instantiate the displacement by $x - x_0$. This removes the final order split and both natural-distance conversions. The `Int.induction_on` pattern already occurs in `WangTiles942SeamColumns`; this proposal specializes that existing induction API, not a new interval-connectivity library.

Incremental benefit. The declaration changes from 31 to 23 nonblank code lines; the proof body changes from 26 to 18. There is no external helper, import, or changed client. The pre-existing `row_reset`, `row_reset_of_north438`, and strip specializations retain the same assumptions and statements. Unlike earlier Nat scanner or finite-cycle propagation proposals, this removes arithmetic transport in a two-sided, strictly bounded integer interval.

Tradeoff and gate. The induction case binder types and normalization in each `convert` must be checked by Lean. Test intervals with one interior point, negative endpoints, and seeds adjacent to either boundary. No finiteness, periodicity, unique Start, or machine-run premise is introduced. A proposed replay module and a proof-body-only patch are in `rows/`.

7 R2 Describe both horizontal edges by one pulse

Source anchors. `WangTiles942StepConstruction.lean` lines 116–148. The current west and east theorems duplicate the same old-head/new-head/direction case proof. The horizontal theorem then redoes direction-dependent arithmetic. Give the single nonzero bond a spatial coordinate instead: index each bond by its *left endpoint*.

For a right move, the bond begins at the old head. For a left move, it begins at the new head. The value is the actual action's rail, not a new transition relation. In the following excerpt, `a` is the existing literal-table action at `sourceIndex cfg q`.

```
def bondSignal (cfg : Config 215) (q : Fin 215) (x : ℤ) : ℕ :=
  let a := WangTiles942.action (sourceIndex cfg q)
  if x = (if a.2.1 then cfg.head else nextHead cfg q)
    then rail a.2.1 a.2.2 else 0
```

Prove a *paired* edge theorem from `stepTile`, `transitionRow`, and `receiveTile`, not from a horizontal-compatibility assumption:

```
  (stepTile cfg q x).west = bondSignal cfg q (x-1) ∧
    (stepTile cfg q x).east = bondSignal cfg q x
```

The proof first uses `nextHead_eq` to identify the west endpoint after shifting by one. It then performs the existing finite constructor case split *once* for the pair of edge equations. The west/east projections give:

```
theorem stepTile_horizontal (cfg : Config 215) (q : Fin 215) (x : ℤ) :
    (stepTile cfg q (x+1)).west = (stepTile cfg q x).east := by
  simp only [stepTile_west, stepTile_east, add_sub_cancel_right]
```

The point is not a more aggressive tactic. Both sides now evaluate the same function at the same bond; compatibility follows from the invariant's indexing. The plane orientation remains $\mathrm{west}(x+1) = \mathrm{east}(x)$, including left moves and entry into halt.

Cost and interface. The measured replacement span is 28 to 29 nonblank code lines and 1,912 to 1,553 bytes, with all helper overhead included. This is an architectural alternative, not a LOC reduction. It changes the statement shapes of `stepTile_west` and `stepTile_east`. It leaves the actual constructors and `stepTile` definition unchanged, and preserves the statements of `stepTile_horizontal` and `step_row`. Retaining the old statement shapes as compatibility wrappers would add further cost, which is not hidden in the metric.

Tradeoff and gate. Compile the paired case proof and search all consumers of the two normalized edge APIs. The invariant does not establish row-step semantics by itself; north/south interpretation still uses its existing live-state premises. Earlier rail descriptors and NorthView classify local data, while this result derives an equation across spatial endpoints. Complete proof, candidate module, proposed source, patch, and static metrics are in `rows/`.

8 Validation and adoption gates

8.1 Completed evidence versus checks still needed

Check	Status and scope
Pinned source identity	860 selected source files verified; five BusyBeaver dependencies separately verified
Prior corpus identity	Six placed packets and five fresh incoming archives bound to source evidence; previous study also included
Active change coverage	Full Lean-tree comparison yields 66 paths, exactly cross-checked against active comparison entries
Candidate source metrics	Independently recounted; helper and import costs distinguished
Patch and statement checks	All six patches apply together to a disposable copy; five target headers retained; R2 changes explicit
Lean elaboration and kernel checks	**Not run**; no Lean or Lake runtime available
Axiom footprint comparison	**Not run**; candidate audit commands are supplied, not claimed output
Downstream rebuild and performance	**Not run**; no compile-time or proof-term-size result

The source-only suite passes 48 Python tests and checks four replay probes with nine expected audit endpoints. A separate bounded model check samples 30,960 spatial edge/matching cases and 2,582 interval/predicate/seed instances. Only 84 interval instances satisfy all premises; all satisfy the conclusion. These are finite QA, not Lean proofs or all-integer verification.

8.2 Replay in a complete pinned checkout

Use the repository's `leanprover/lean4:v4.32.0` and locked Mathlib revision `81a5d257c8e4`. The archive is not a standalone build environment. First obtain a *complete* checkout with its submodules and dependencies; do not infer build readiness from this source snapshot.

1. Re-run the snapshot and corpus verification scripts. Inspect each patch against the pin and apply it only in a throwaway worktree. Keep R2 independent from the five local replacements so its API cost is visible.
2. Compile the isolated candidate modules. Confirm they prove their own statements rather than calling the original target theorem. Run every supplied regression statement, especially target 215 versus invalid 216, old/new write positions, time zero, and both movement directions.
3. Rebuild changed source modules and paired Checks modules. For E1 and M1, rebuild the action, step, and row-construction consumers; for R1 rebuild reset and strip clients; for R2 rebuild all users of west/east projection theorems and row construction.
4. Compare `#print axioms` for old and proposed declarations and relevant public endpoints. A shorter tactic proof may introduce standard axioms even when the old proof did not. Do not treat their absence from source text as an axiom audit.
5. Measure elaboration and memory only after successful checking, using comparable clean/cached conditions. Keep the less fragile proof if the shorter one imposes substantial import or simplifier cost.

9 Artifacts and further research questions

9.1 Archive map

Location	Contents and purpose
`report/`	This editable TeX manuscript, PDF, and reproducible XeLaTeX build script
`machine/`	Three isolated candidates, independent proof-body patches, proposed modules, metrics, static validation, pinned absolute-value source
`encoding/`	Sentinel-preserving injectivity candidate and pinned finite-equivalence source evidence
`rows/`	Interval and pulse candidates, proposed modules, patches, metric script and supporting checks
`baseline_audit/`	Prior-corpus identity records, full assumed-applied API ledger, conflict/novelty decisions and audit scripts
`validation/`	Replay plan, isolated-probe runner, source-only audit, 48-test suite, explicit unavailable-runtime preflight
`provenance/`	Source manifest, full Lean-tree delta, 27-commit timeline, metric and verification evidence
`sources/` and `dependencies/`	Source snapshot and separately verified direct BusyBeaver imports; not a complete build closure

The source links in this report all target the reviewed pin or the pinned Mathlib dependency, rather than moving `main`. The source manifests distinguish reused, hash-matched bytes from fresh downloads. The prior ledger preserves exact packet/section locators so an apparent novelty claim can be audited without accepting a keyword-only search.

9.2 Questions worth resolving next

Does E1 pay for its import? Its mathematical reuse is exact, but the explicit finite-equivalence import can cost more than a small local injectivity proof. Compile and profile both versions before adopting it. A new project-wide encoding abstraction is not justified by this single client.

Does R2 have more consumers? Its strongest case is maintainability: one spatial edge invariant replaces duplicate proofs and exposes why matching works. If no subsequent construction uses that view, keeping the old public west/east statements may be preferable. Search actual consumers first; do not count hypothetical applications as savings.

How stable are the two normalization sites? The finite-equivalence identity uses `simp`; the interval proof uses `convert` and `omega`. Lean replay should establish whether explicit simp lemmas or small arithmetic equalities improve robustness without restoring the old scaffolding.

What remains beyond already proposed abstractions? Future work should identify a concrete proof obligation that survives the ledger. A shared trace-envelope record, a broader controller simulation, or another generated-certificate layer is not recommended here: previous proposals already cover much of that territory, and no additional current consumer saving has been demonstrated.